

Java Cheatsheet

Java Interview Prep — 2026 Edition · Learn & Excel

One printable page. Read it once before walking in. Don't try to memorize — trust your prep.

OOP recap

Concept	Rule
Overloading	Same method name, different parameters. Compile-time, in same class.
Overriding	Same signature in subclass. Runtime polymorphism. <code>@Override</code> .
Abstract class	Can have state + concrete methods. Extend with <code>extends</code> . Single inheritance only.
Interface	No state (only constants). <code>default</code> methods since Java 8. Multiple implementation via <code>implements</code> .
Access modifiers	<code>public</code> everywhere · <code>protected</code> same package + subclasses · default same package · <code>private</code> same class
final	variable = no reassign · method = no override · class = no extend
static	belongs to class, not instance · cannot use <code>this</code>
equals/hashCode	<code>a.equals(b)</code> → must have <code>a.hashCode() == b.hashCode()</code> . Override both or neither.

Collections — pick the right one

Need	Use
Indexed list, fast random access	<code>ArrayList</code>
Frequent middle insert/remove (with iterator)	<code>LinkedList</code>
LIFO stack	<code>Deque<T> s = new ArrayDeque<>();</code> (NOT <code>Stack</code>)
FIFO queue	<code>Queue<T> q = new ArrayDeque<>();</code>
Producer-consumer	<code>BlockingQueue</code> (e.g. <code>ArrayBlockingQueue</code>)
Unique elements, no order	<code>HashSet</code>
Unique + sorted	<code>TreeSet</code>
Unique + insertion order	<code>LinkedHashSet</code>
Key-value, no order	<code>HashMap</code>
Key-value sorted by key	<code>TreeMap</code>
Key-value, insertion/access order	<code>LinkedHashMap</code> (set <code>accessOrder=true</code> for LRU)
Concurrent map	<code>ConcurrentHashMap</code>
Read-heavy concurrent list	<code>CopyOnWriteArrayList</code>

Java 8+ Map idioms: `map.getOrDefault(k, d)` · `map.putIfAbsent(k, v)` · `map.computeIfAbsent(k, key -> compute(key))` · `map.merge(k, 1, Integer::sum)`

Concurrency primitives — when each

Tool	Use when
<code>volatile</code>	Visibility only (flag read by many, written rarely). NOT for compound ops.
<code>synchronized</code>	Mutual exclusion + visibility. Default choice for compound ops on shared state.
<code>AtomicInteger</code> , <code>AtomicLong</code> , etc.	Lock-free CAS for single-variable read-modify-write.
<code>LongAdder</code>	High-contention counter. Faster than <code>AtomicLong</code> under heavy writes.
<code>ReentrantLock</code>	Need <code>tryLock</code> , fairness, interruptible acquire, or multiple <code>Condition</code> s.
<code>ReadWriteLock</code>	Many readers, few writers.
<code>StampedLock</code> (Java 8+)	Read-heavy, want optimistic reads.
<code>BlockingQueue</code>	Producer-consumer with backpressure.
<code>ExecutorService</code>	Thread pooling. Use <code>ThreadPoolExecutor</code> directly in prod (NOT <code>newFixedThreadPool</code>).
<code>CompletableFuture</code>	Async composition. Chain with <code>thenCompose</code> / <code>thenCombine</code> / <code>exceptionally</code> .
Virtual threads (Java 21)	IO-bound server work. <code>Executors.newVirtualThreadPerTaskExecutor()</code> .

Always wait in a loop: `while (!conditionMet()) cond.await();` — never `if`. Spurious wakeups exist.

Stream API — quick patterns

```
// Assumes: import static java.util.stream.Collectors.*;

// Filter + map + collect to list
List<String> emails = users.stream()
    .filter(User::isActive)
    .map(User::getEmail)
    .toList(); // Java 16+

// Group by + count
Map<String, Long> byDept = employees.stream()
    .collect(groupingBy(Employee::getDept, counting()));

// Sum + average in one pass
DoubleSummaryStatistics stats = orders.stream()
    .mapToDouble(Order::getAmount)
    .summaryStatistics(); // get sum, avg, min, max, count

// Find first match
Optional<User> admin = users.stream()
    .filter(u -> u.getRole() == ADMIN)
    .findFirst();

// String concatenation
String csv = items.stream()
    .map(Item::getName)
    .collect(joining(", ", "[", "]"));
```

Lazy reminder: nothing runs until a terminal op (`collect` / `forEach` / `count` / `reduce` / `findFirst`).

Exception handling

```
// try-with-resources — auto-close, suppressed exception handling
try (var conn = DriverManager.getConnection(url);
    var stmt = conn.prepareStatement(sql)) {
    stmt.executeUpdate();
} // close in reverse order, suppressed exceptions chained

// Multi-catch
try { ... }
catch (IOException | SQLException e) { ... }
```

Checked vs unchecked: checked = caller MUST handle (compile-enforced). Unchecked = bug, propagates freely. Modern Java leans unchecked; Spring wraps `SQLException` → `DataAccessException`.

JVM commands you'll be asked about

Command	Use
<code>jps -lv</code>	List running JVMs with main class + args
<code>jstack <pid></code>	Thread dump — find deadlocks, blocked threads
<code>jmap -heap <pid></code>	Heap summary
<code>jmap -histo:live <pid></code>	Live objects by class (count + bytes)
<code>jcmd <pid> GC.heap_dump file.hprof</code>	Heap dump for offline analysis (MAT, VisualVM)
<code>jstat -gc <pid> 1s</code>	Live GC stats every 1 sec
<code>-Xlog:gc*</code> (Java 9+)	Enable GC logging (unified logging — replaces <code>-XX:+PrintGCDetails</code> from Java 8)
<code>-XX:MaxGCPauseMillis=200</code>	Tune G1 pause target

Spring annotations — what each is for

Annotation	Purpose
<code>@Component</code>	Generic Spring-managed bean
<code>@Service</code>	Business logic layer (semantic, no behavioral diff vs Component)
<code>@Repository</code>	Data-access layer; auto-wraps JDBC/JPA exceptions as <code>DataAccessException</code>
<code>@Controller</code>	Spring MVC controller (returns view name)
<code>@RestController</code>	<code>@Controller</code> + <code>@ResponseBody</code> (returns body as JSON/XML)
<code>@Autowired</code>	DI injection (constructor injection preferred — no annotation needed if single constructor)
<code>@Qualifier("name")</code>	Disambiguate when multiple beans match the type
<code>@Configuration</code> + <code>@Bean</code>	Programmatic bean definitions
<code>@Transactional</code>	Declarative transaction boundary
<code>@Value("\${prop}")</code>	Inject property from application.properties / env
<code>@ConditionalOn*</code>	Conditional bean registration (auto-config use)

Spring Boot 3 = `jakarta.*` (not `javax.*`). Migration is mechanical: search/replace + verify.

Java 21 new keywords + syntax

```
// Records (Java 16) – immutable data class, one line
public record Point(int x, int y) {}

// Sealed (Java 17) – restricted inheritance
public sealed interface Shape permits Circle, Square {}
public record Circle(double r) implements Shape {}
public record Square(double side) implements Shape {}

// Pattern matching switch (Java 21) – exhaustive + binding
String describe(Shape s) {
    return switch (s) {
        case Circle(double r) -> "circle radius " + r;
        case Square(double side) -> "square side " + side;
    };
}

// var (Java 10) – local type inference
var users = new ArrayList<User>();           // type inferred

// Text blocks (Java 15)
String sql = """
    SELECT id, name
    FROM users
    WHERE active = true
    """;

// Virtual threads (Java 21)
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    executor.submit(() -> handleRequest());
}
```

Interview-day micro-tips

- **Read the question twice** before talking. Most candidates rush.
- **State your assumptions out loud** before coding. Interviewers grade thinking, not typing speed.
- **SELECT * and String.equals (without Objects.equals) signal junior-level.** Avoid both.
- **When stuck**, say "I'd want to verify X with the docs, but my first instinct is..." — beats silence or guessing.
- **Always mention edge cases** even if not asked: null, empty, concurrent access, large input.
- **End your answer**, don't trail off. "...and that's why I'd pick the Stream version here." Strong close = remembered.

Print this. Keep it folded in your bag. You're ready.